Git & GitHub
Complete Handbook

Version Control for a Better Tomorrow!

1. What is Git?

- Git is a Distributed Version Control System (DVCS).
- Tracks changes in your code over time.
- Helps multiple developers work together efficiently.
- Lets you go back to previous versions (history).
- Fast, lightweight and reliable.

2. What is GitHub?

- GitHub is a cloud platform for Git repositories.
- Hosts Git repositories online.
- Enables collaboration, code review and project management.
- Provides tools like Issues, Pull Requests, Actions and more.

3. Why Use Git & GitHub?

- ✓ Track and manage changes in your code.
- ✓ Collaborate with others seamlessly.
- ✓ Work on multiple features/branches.
- ✓ Recover from mistakes (revert to old versions).
- ✓ Build better software, faster.

4. Key Concepts

- Repository (Repo): A project folder with all your code and history.
- Commit: A snapshot of your changes.
- Branch: A separate line of development.
- Merge: Combine changes from one branch to another.
- Pull / Push: Get changes from / send changes to a remote repository (e.g., GitHub).

5. Git vs GitHub

Git	GitHub
A tool (VCS)	A platform (Cloud)
Works on your local machine	Hosts Git repositories
Handles version control	Enables collaboration & sharing
Open-source	Cloud-based service
You install it	No installation (use online)

6. Get Started

- ✓ Install Git (https://git-scm.com).
- ✓ Create a GitHub account (https://github.com).
- ✓ Set up your SSH keys (for secure access).
- ✓ Initialize a repository and make your first commit!

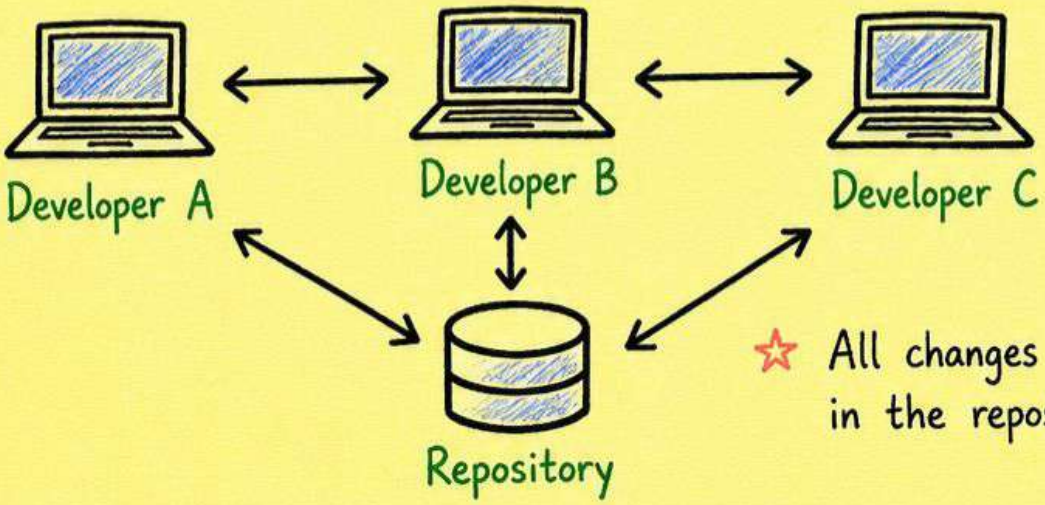
♡ Practice. Build. Collaborate. Grow! ♡

Git & GitHub

Complete Handbook

1. What is Git?

- Git is a Distributed Version Control System (DVCS).
- It tracks changes in your code over time.
- It allows multiple developers to work together on a project efficiently.
- Created by Linus Torvalds in 2005.



☆ All changes are stored in the repository.

2. Why use Git?

- ✓ Keep history of every change.
- ✓ Work offline and commit later.
- ✓ Easy collaboration with teams.
- ✓ Branching & merging makes development safe.
- ✓ Backup of your code (no more "lost code" ☹)

In Short:
Git helps you track, manage and collaborate on code like a pro!
☆

3. Git Architecture & Areas

Git has 3 main areas where your files exist.

1. Working Directory

This is your project folder on your computer.

git add →

← git restore

2. Staging Area

A preparation area where you select changes to be committed.

git commit →

← git reset

3. Repository (.git)

This is where Git stores the committed changes permanently.

Flow: Working Directory —git add→ Staging Area —git commit→ Repository

4. File Status in Git

Git tracks the status of your files.

- **Untracked:** New file that Git doesn't know about. — Solution: git add <file>
- **Staged:** Changes added to staging area. — Solution: git commit
- **Modified:** Changes made but not staged yet. — Solution: git add <file>
- **Committed:** Changes safely stored in repository. — No action needed ☺

5. First Time Setup

```
$ git config --global user.name "Your Name"
$ git config --global user.email "you@example.com"
$ git config --global core.editor "code --wait"
$ git config --global --list        # to see config
```

Do this once so Git knows who you are! ☺

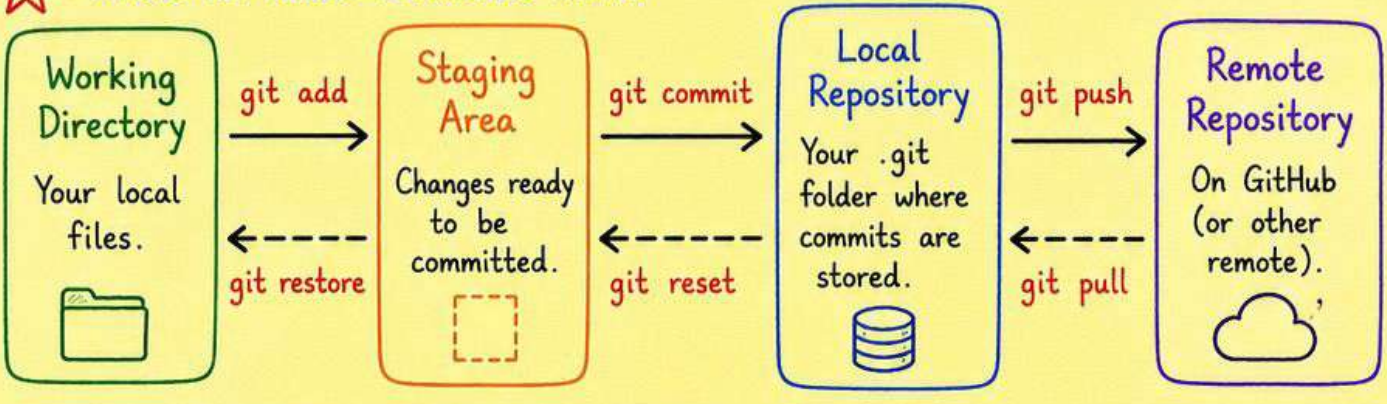
Pro Tip: Use meaningful commit messages.
Example: "Add login feature" ✓

6. Essential Git Commands (I)

Here are some must-know Git commands you'll use every day!

Command	Description
git init	Initialize a new Git repository. .git
git clone <url>	Clone (copy) a remote repository.
git status	Check the status of your working directory and staging area.
git add <file>	Add file contents to the staging area.
git add .	Add all changes in the current directory to the staging area.
git commit -m "msg"	Commit staged changes with a message.
git log	View commit history.
git diff	Show changes between commits, files or working directory.
git branch	List all branches in the repository.
git checkout <branch>	Switch to another branch.
git pull	Fetch and merge changes from remote repository.
git push	Push your changes to remote repository.

Where do these commands work?



7. Git Workflow

This is the basic workflow every time you make changes.

1. Modify
Make changes in files

→

2. Stage
Add changes to staging area

`git add <file>`
or
`git add .`

→

3. Commit
Save changes to local repo

`git commit -m "message"`

→

4. Push
Send changes to remote repo

`git push origin <branch>`

→

Remote Repository (GitHub)

File Status Overview

Status	Meaning	Example
Untracked	Git doesn't know about this file yet.	New file created.  new.txt
Staged	Changes added to staging area, ready to commit.	Added with git add. 
Modified	File changed but not added to staging.	You edited the file. 
Committed	Changes saved inside local repository.	Saved with git commit. 

Common Daily Workflow

Edit files (Modify)

→

Stage changes (git add)

→

Commit changes (git commit)

→

Push to remote (git push)

→

Team gets updates

Best Practices

- ✓ Commit small and meaningful changes.
- ✓ Write clear commit messages.
- ✓ Pull latest changes before you start working.
- ✓ Use .gitignore to ignore unnecessary files.

Quick Tip

Think before you commit.
A good commit message today saves confusion tomorrow!

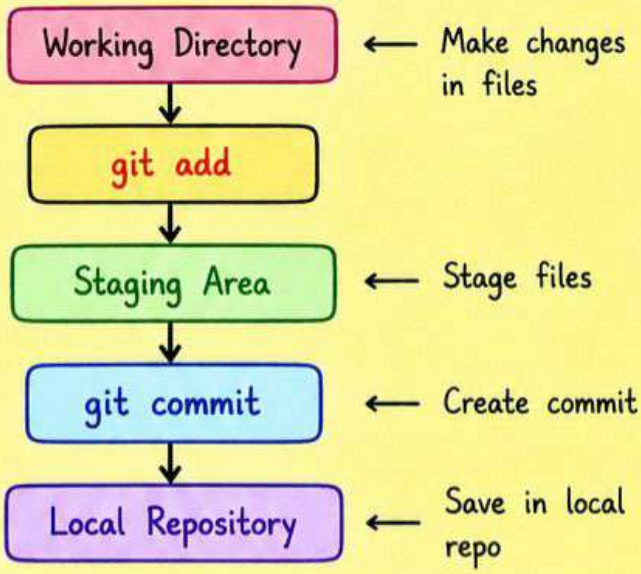
7. Git Basics

Understand the most important Git commands.

Basic Commands

`git init`	:	Initialize a new repo
`git clone`	:	Clone a remote repo
`git status`	:	Check status of changes
`git add`	:	Stage changes
`git commit`	:	Commit changes
`git log`	:	View commit history
`git diff`	:	Show differences
`git branch`	:	List / create branches
`git checkout`	:	Switch branches/commits
`git merge`	:	Merge branches

Commit Flow



Branching

- A branch is a different line of development.
- Helps work on features independently.
- Default branch is usually `main`.
- Merge branches to combine changes.

Common Branch Commands

`git branch`	:	List all branches
`git branch <name>`	:	Create a new branch
`git checkout <name>`	:	Switch to a branch
`git merge <name>`	:	Merge branch into current
`git branch -d <name>`	:	Delete a branch

Working with Remotes

`git remote -v`	:	List remote repos
`git remote add origin <url>`	:	Add remote repo
`git push origin main`	:	Push changes
`git pull origin main`	:	Pull changes
`git fetch`	:	Fetch changes
`git clone <url>`	:	Clone a repo

Undo Changes

`git reset <file>`	:	Unstage a file
`git reset --hard`	:	Discard all changes
`git revert <commit>`	:	Undo a commit
`git checkout -- <file>`	:	Discard changes in a file

Useful Tips

- ✓ Use meaningful commit messages.
- ✓ Pull before pushing to avoid conflicts

Master Git basics, and you'll never fear

8. Branching in Git

Branches allow you to work on features or fixes independently without affecting the main codebase.

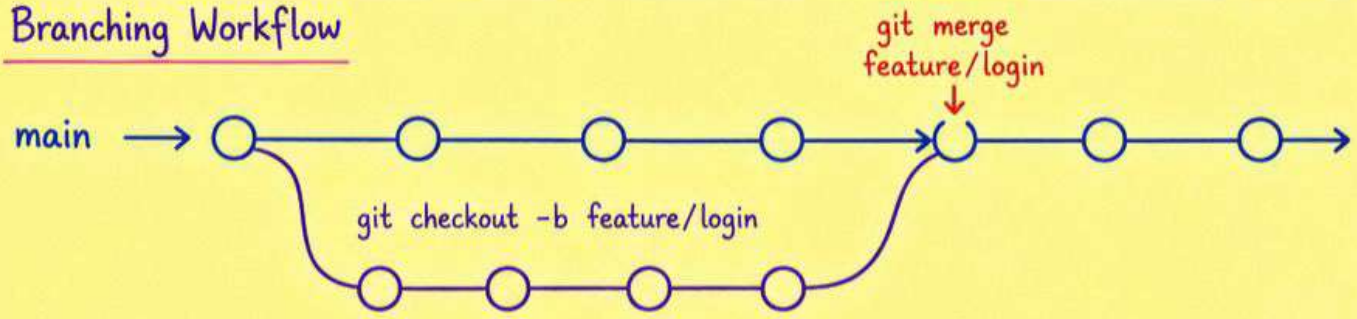
Why use branches?

- Work on new features without breaking main.
- Fix bugs safely.
- Collaborate with team smoothly.
- Experiment freely.

Common Branch Types

- main / master : Production ready code.
- develop : Integration branch for features.
- feature/* : New features.
- bugfix/* : Bug fixes.
- hotfix/* : Urgent fixes in production.
- release/* : Preparing for a new release.

Branching Workflow



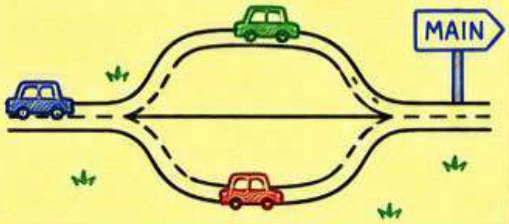
Create branch → Work & commit → Merge back to main

Important Branch Commands

Command	Description
git branch	List all local branches.
git branch <name>	Create a new branch.
git checkout <name>	Switch to a branch.
git checkout -b <name>	Create and switch to a new branch.
git branch -d <name>	Delete a branch (safe delete).
git branch -D <name>	Force delete a branch.
git merge <name>	Merge a branch into current branch.
git branch -a	List all branches (local + remote).

Visual Analogy

Think of branches like roads. You can explore new paths, and come back to the main road anytime.



Pro Tip

Keep your main branch clean. Always create a branch for every new task.

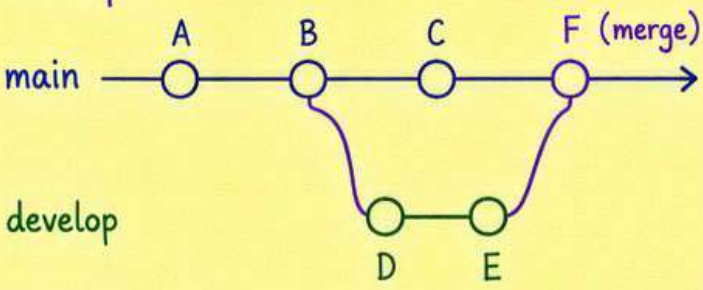
9. Merging in Git

Merging combines changes from one branch into another.

How Merge Works

1. You are on the target branch (usually main).
2. Git finds the common ancestor.
3. Git applies the changes from the source branch.
4. A new merge commit is created.

Example



A, B, C : Commits on main
D, E : Commits on develop
F : Merge commit

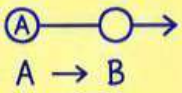
Merge Command

```
# Step 1: Switch to target branch
git checkout main

# Step 2: Merge source branch
git merge develop
```

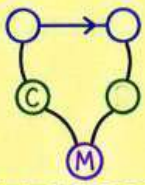
```
# With message
git merge develop -m "Merging develop into main"

# Fast-forward only (no merge commit)
git merge --ff-only develop
```

Merge Types

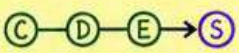
- **Fast-Forward Merge**
  Moves the branch pointer forward. (No merge commit created)



- **Merge Commit**
  Creates a new commit to join branches.



- **Squash Merge** (on GitHub)
  Combines all commits into one.



While Merging – Conflicts

If Git can't automatically merge changes, you will get a conflict.

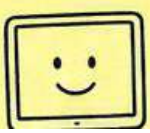
1. Git marks conflicted files.
2. Open the files and fix conflicts.
3. Stage the resolved files.
4. Commit the merge.

Commands

```
git status                  # see conflicted files
# After resolving conflicts
git add <file>              # stage resolved files
git commit                  # complete the merge
```

Best Practices

- ✓ Pull latest changes before merging.
- ✓ Resolve conflicts carefully.
- ✓ Test your code after merge.



Clean merges, happy code!

Pro Tip

Keep your branches small and focused. It makes merges easier and

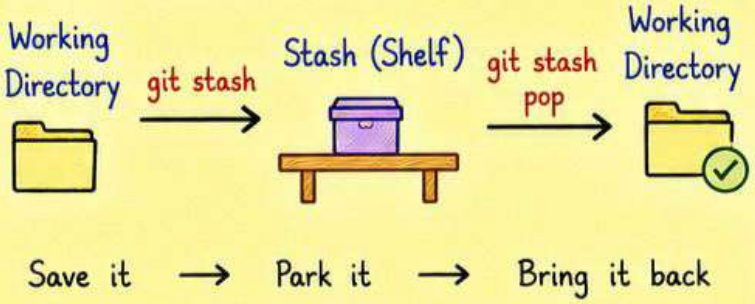
10. Git Stash

Stash lets you save your uncommitted changes temporarily and work on something else.

When to use Stash?

- You are mid-way but need to switch branches.
- You want a clean working directory.
- You are pulling latest changes but have local work.

Basic Stash Workflow



Important Stash Commands

Command	Description	Example
git stash	Stash your changes (tracked files).	git stash # Stash changes
git stash save "msg"	Stash with a message.	git stash save "WIP: login page" # Add message
git stash list	List all stashes.	git stash list # Show stash list
git stash show	Show changes in latest stash.	git stash show # Show diff of latest
git stash show -p	Show patch of latest stash.	git stash show -p # Detailed diff
git stash apply	Apply latest stash (keep it).	git stash apply # Apply without removing
git stash pop	Apply latest stash and remove it.	git stash pop # Apply and remove
git stash drop	Remove latest stash.	git stash drop # Delete latest stash
git stash clear	Remove all stashes.	git stash clear # Clear all stashes
git stash apply stash@{n}	Apply specific stash.	git stash apply stash@{2} # Apply stash #2

Example Walkthrough

① You have changes in files.	(modify code...)
② Stash your work.	git stash
③ Switch to another branch.	git checkout feature
④ Do your work and commit.	git commit -m "done"
⑤ Come back to original branch.	git checkout main
⑥ Bring back your stashed work.	git stash pop
⑦ Continue where you left off!	

Stash List Example

git stash list

```
stash@{0}: On main: WIP: login page
stash@{1}: On main: fixed navbar
stash@{2}: On develop: refactor code
```

Tip:
Stashes are stack based.
stash@{0} is the most recent.

Best Practices

- ✓ Keep stash list clean. Clear old stashes.
- ✓ Use meaningful messages.

Note

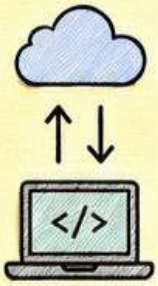
Stash works only on local repository.

11. Remote Repositories

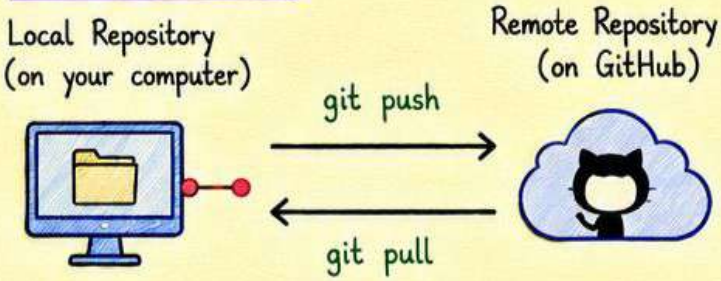
A remote repository is a version of your project that is hosted on the internet (like GitHub). You can collaborate and share your code.

Why use Remotes?

- Back up your code.
- Collaborate with others.
- Work from anywhere.
- Share and contribute to open source.



Local vs Remote



Push → send changes to remote
Pull → get changes from remote

Important Remote Commands

Command	Description	Example	Notes
git remote -v	List all remote repositories.	git remote -v	Shows fetch & push URLs
git remote add <name> <url>	Add a new remote.	git remote add origin https://github.com/user/repo.git	"origin" is a common remote name
git remote remove <name>	Remove a remote.	git remote remove origin	Removes the remote
git fetch <name>	Download changes from remote (no merge).	git fetch origin	Fetch latest changes
git pull <name> <branch>	Fetch & merge changes from remote.	git pull origin main	Get latest & update local branch
git push <name> <branch>	Push local changes to remote.	git push origin main	Upload your commits
git clone <url>	Clone a remote repository.	git clone https://github.com/user/repo.git	Creates local copy

Typical Collaboration Workflow

1. Clone the repository — `git clone <url>`
2. Create a new branch — `git checkout -b feature/login`
3. Make changes and commit — `git add .` / `git commit -m "Add login page"`
4. Push your branch — `git push origin feature/login`
5. Open a Pull Request on GitHub → Ask to merge
6. After review & approval, your code is merged!

What is "origin"?

- It is just a default name for your remote.
- You can name it anything you want.

Example:

```
git remote add myRepo https://github.com/user/repo.git
git push myRepo main    # using custom name
```

Tip:
Always pull before you push to avoid conflicts.
`git pull origin main`

Good to Know

- .git folder stores remote info in: .git/config

Pro Tip

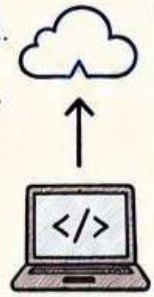
Keep your local main branch up to date.

12. GitHub Basics

GitHub is a platform to host Git repositories in the cloud.
It helps you collaborate, share and build better software.

Why use GitHub?

- Store your code safely in the cloud.
- Access your project from anywhere.
- Collaborate with your team.
- Track issues and manage tasks.
- Showcase your work to the world.



GitHub vs Local Git

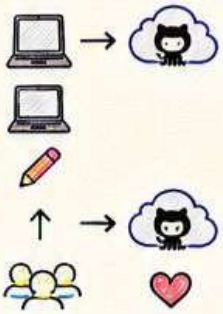
Think: Local is your workspace,
GitHub is your backup and collaboration hub.

Important GitHub Commands

Command	Description	Example	Notes
git init	Initialize a new local repository.	git init	Creates .git folder
git remote add origin <url>	Connect local repo to GitHub.	git remote add origin https://github.com/user/repo.git	origin is default name
git branch -M main	Rename current branch to main.	git branch -M main	-M forces rename
git push -u origin main	Push code to GitHub and set upstream.	git push -u origin main	First push to GitHub
git pull origin main	Pull latest changes from GitHub.	git pull origin main	Fetch + merge
git status	Check the status of changes.	git status	Shows modified files
git log --oneline	Show commit history in short.	git log --oneline	Clean and compact view
git remote -v	List all remote repositories.	git remote -v	-v shows URLs

Typical GitHub Workflow

1. Create a repo on GitHub.
2. Clone the repo to your computer.
3. Make changes and commit locally.
4. Push your changes to GitHub.
5. Collaborate, review, and improve!



Repository Visibility

Public Repository

- Anyone can see your code.
- Great for open source projects.

Private Repository

- Only you and people you invite can see.
- Best for personal or company code.

Useful GitHub Features

- **Issues** – Track bugs, features, and tasks.
- **Pull Requests** – Review and merge changes.
- **Actions** – Automate your workflow (CI/CD).

Pro Tip

Always write a good README.md file.
It explains your project and makes it easy for others to understand.

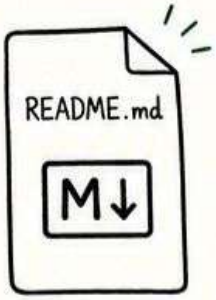
13. GitHub Essentials

Some must-know concepts to make your GitHub profile and projects look professional.

1. README.md

The first thing people see in your repo!

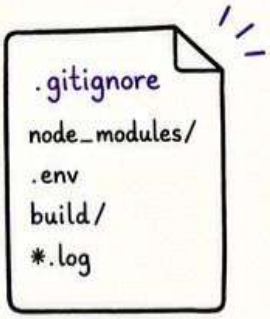
- Explain what your project does.
- How to install and run.
- Tech stack used.
- Screenshots / demos (if any).
- How others can contribute.



2. .gitignore

Tells Git which files / folders to ignore.

- Keeps your repo clean.
- Avoids pushing sensitive or unnecessary files.
- Common ignores:
  - node_modules/
  - .env
  - build/
  - *.log



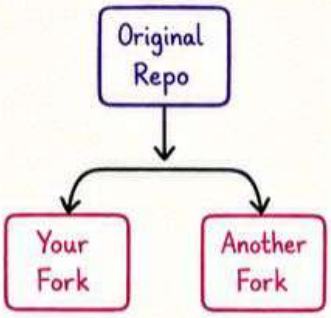
3. Stars and Forks

Star

- Like / bookmark a repo.
- Shows appreciation.
- Helps you find it later.

Fork

- Create a copy of someone's repo.
- Make changes without affecting the original.
- Used in open source contributions.



4. Issues

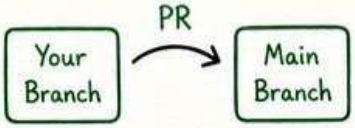
Track bugs, tasks, or feature requests.

- Open an issue.
- Discuss and collaborate.
- Assign, label and close.
- Great for project management!

5. Pull Requests (PR)

Propose your changes to the original project.

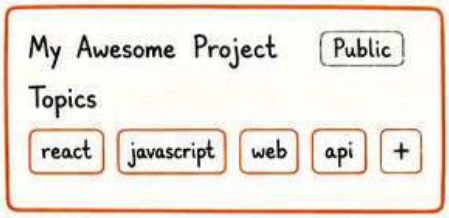
- Create a PR from your fork (or branch).
- Discuss the changes.
- Review and approve.
- Merge when ready!



6. Repository Topics

Topics help others discover your project.

- Add relevant topics to your repo.
- Use keywords related to your project.
- Example: react, javascript, web, api



Tip

Good topics = More visibility for your project!

7. GitHub Profile Tips

- ✓ Add a profile picture.
- ✓ Write a short and clear bio.
- ✓ Pin your best repositories.

8. Useful Shortcuts

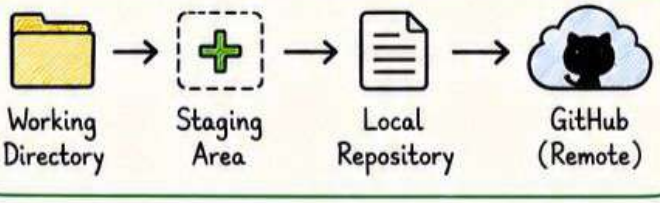
- `git remote -v` → Show remotes
- `git status` → Check status
- `git add .` → Add all changes

14. Summary & Final Tips

You now know the basics of Git & GitHub! Here is a quick recap and final tips to keep in mind.

Git Workflow Recap

1. Make changes in your working directory.
2. Stage the changes: `git add .`
3. Commit the changes: `git commit -m "message"`
4. Push the changes to GitHub: `git push origin main`



Key Takeaways

- Git tracks changes in your code.
- Commits are snapshots of your work.
- Branches help you work on features without breaking main.
- Remotes (like GitHub) help you collaborate and backup.
- Pull before push to avoid conflicts.
- Write clear commit messages.
- Keep your repository clean and organized.

Most Used Commands At a Glance

Category	Commands	Category	Commands
Setup	`git init` `git clone <url>`	Pull / Fetch	`git pull origin <branch>` `git fetch origin`
Staging	`git add <file>` `git add .`	Branching	`git branch` `git checkout -b <branch>`
Commit	`git commit -m "message"` `git commit -am "message"`	Merge	`git merge <branch>`

Push	`git push origin <branch>`
	`git push -u origin <branch>`
		Undo / Revert	`git reset --soft HEAD~1`
			`git reset --hard HEAD~1`
			`git revert <commit>`

Common Mistakes

- ✗ Pushing without pulling first.
- ✗ Committing without meaningful message.
- ✗ Pushing sensitive files (.env, node_modules).
- ✗ Working on main branch directly.
- ✗ Forgetting to .gitignore unnecessary files.

.gitignore Example

```
node_modules/
.env
build/
dist/
*.log
.DS_Store
coverage/
```

Keep your repo clean & secure!

Best Practices

- ✓ Pull latest changes before starting work.
- ✓ Create a new branch for each feature or fix.
- ✓ Write small, focused commits.
- ✓ Review your code before pushing.
- ✓ Keep your README updated.

15. Resources & Next Steps

Keep learning and keep building! Here are some helpful resources and next steps for your Git & GitHub journey.

Recommended Resources

Pro Git Book
By Scott Chacon & Ben Straub
(git-scm.com/book)

YouTube Channels
freeCodeCamp, Traversy Media, Programming with Mosh

Official Documentation
git-scm.com/docs
docs.github.com

Interactive Practice
learngitbranching.js.org
github.com/explore

Cheat Sheets
github.com/tiimgreen/github-cheat-sheet

Practice Ideas

- Create a new repository and push your first project.
- Work on a small project using branches and pull requests.
- Collaborate with a friend on GitHub.
- Raise and fix issues in open source projects.
- Explore GitHub Actions and automate your workflow.
- Keep building real projects and share your work!

Git & GitHub Roadmap

1. **Basics**
Learn Git basics, commands and workflow.

2. **Practice**
Practice daily, build small projects.

3. **GitHub**
Host your projects, collaborate and explore.

4. **Contribute**
Contribute to open source and help the community.

5. **Grow**
Build real projects, keep learning, keep improving!

Handy Tips

- ✓ Commit often with meaningful messages.
- ✓ Use .gitignore to keep your repo clean.
- ✓ Work on feature branches. Keep main branch stable.
- ✓ Pull before push. Stay updated!
- ✓ Read, learn and practice regularly. Consistency is the key!

Essential Files to Remember

- README.md → Project information
- .gitignore → Ignore files/folders
- LICENSE → License information
- CONTRIBUTING.md → How others can contribute
- CODE_OF_CONDUCT.md → Community guidelines

Great Developers Use Git!

- ☆ Git saves time.
- ☆ Git prevents mistakes.
- ☆ Git makes teams stronger.
- ☆ Git helps you grow.
- ☆ Git is a skill that stays with you!

Final Words

Git is about version control.
GitHub is about collaboration.
Together, they help you build better software.

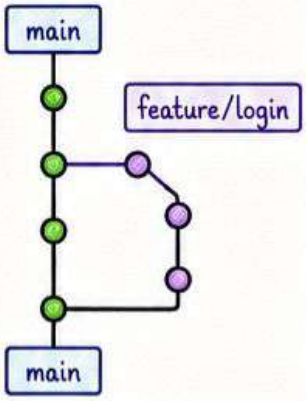
One day or day one.

16. Advanced Topics

Once you are comfortable with the basics, you can explore advanced features to level up!

1. Branching Strategies

- Feature Branch Workflow
- Gitflow Workflow
- Trunk Based Development
- Choose what works best for your team!



2. Rebasing vs Merging

Merge

Keeps the history as it is. Creates a merge commit.

Rebase

Reapplies your commits on top of the latest changes.

Use Merge for shared history.
Use Rebase for clean history (before pushing).

3. GitHub Powerful Features

Pull Requests

- Propose changes
- Discuss and review
- Merge with confidence

Code Reviews

- Get feedback
- Improve code quality
- Learn from others

GitHub Actions

- Automate workflows
- CI / CD
- Save time and effort

GitHub Packages

- Host packages
- Private & public
- Easy to use

Insights

- Track progress
- See contributors
- Understand repo activity

These features help you build better, ship faster and work like a pro!

4. Useful Git Aliases

```
alias st = status
         # git status
alias co = checkout
         # git checkout
alias br = branch
         # git branch
alias ci = commit
         # git commit -m
alias lg = log --oneline
         --graph --all
         # nice log view
```

```
$ git lg
* e3f1a2d Update README
* b7c9d1a Fix login bug
a1b2c3d Add login page
9f8e7d6 Initial commit
```

Add these in your ~/.gitconfig to save time every day!

5. Keeping Your Repo Clean

- ✓ Write a clear README.md
- ✓ Use meaningful commit messages
- ✓ Keep .gitignore updated
- ✓ Follow coding standards
- ✓ Remove unused branches
- ✓ Keep history clean
- ✓ Document your project

A clean repo shows professionalism!
Be proud of your work.

6. Debugging Git Issues

- git status → See what's going on
- git log → Check history
- git diff → See changes
- git reflog → Recover lost commits

7. Where to Go Next?

- Build real projects
- Contribute to open source
- Read and explore documentation

17. Final Summary

Git is local. GitHub is remote.
Together they make you a better developer!

Git vs GitHub (Quick Recap)

Git

- Distributed VCS
- Works locally
- Tracks changes
- Commits, branches, merge, etc.
- No user accounts

GitHub

- Cloud platform
- Hosts Git repositories
- Collaboration & sharing
- Pull Requests, Issues, Actions, Projects
- User accounts

Complete Workflow

1. Code — Make changes locally
2. Stage — git add .
3. Commit — git commit -m "msg"
4. Push — git push origin main
5. GitHub — Share, review and collaborate

Always Pull before Push to keep your work up to date and avoid conflicts!

Important Commands Cheat Sheet

Setup

- git init → New local repo
- git clone <url> → Clone remote repo

Daily Work

- git add . → Stage all changes
- git commit -m "msg" → Commit changes
- git status → Check status

Sharing & Update

- git push origin <branch> → Push changes
- git pull origin <branch> → Fetch + merge
- git fetch origin → Fetch latest changes

Branching

- git branch → List branches
- git branch <name> → Create branch
- git checkout <branch> → Switch branch
- git merge <branch> → Merge branch

Undo / Fix

- git log --oneline → History (short)
- git reset --hard <commit> → Undo to commit
- git revert <commit> → Safe undo
- git stash → Save changes temporarily

Key Concepts to Remember

Repository
A project folder tracked by Git.

Branch
A parallel version of your code.

Commit
A snapshot of your changes.

Remote
A version of your repo on GitHub.

Pull Request
Propose changes and review.

Small commits
Every day
Big impact
One day!

Best Practices

- [x] Write clear and meaningful commit messages.
- [x] Pull latest changes before starting work.
- [x] Create a new branch for each feature or fix.
- [x] Keep your branches small and focused.
- [x] Review your code before pushing.
- [x] Keep README.md updated.
- [x] Use .gitignore to ignore unnecessary files.
- [x] Collaborate, communicate and help others.

Good habits make you a great developer!

Keep Going!

- Explore GitHub Actions and automate workflows.
- Contribute to open source projects.

MASTER GIT & GITHUB

> The best way to predict the future